PROGRAMMING IN C • STUDY GUIDE

ARRAYS

Concepts, Syntax, Memory Model & Solved Examples

University Exams | Competitive Exams | Teacher Recruitment Tests

```
int marks[5] = {80, 90, 75, 60, 95};
```

A Complete Revision Resource

Chapter 1: Introduction to Collections in C

1.1 Concept of a Collection

> **DEFINITION**
>
> A **collection** is a programming construct that groups multiple data items together under a single name so that they can be stored, accessed, and manipulated as one logical unit, instead of declaring a separate variable for every value.

In real life, a collection is like a row of lockers sharing one locker-room name, where each locker still has its own number. In programming, when a problem requires handling many values of a related kind — such as marks of 50 students or temperatures of 12 months — using individual variables becomes impractical. Collections solve this by providing a single, organized container for related data.

Need for Collections in Programming

- Avoids declaring hundreds of individual variables for similar data.
- Enables use of loops to process large amounts of data efficiently.
- Provides a logical grouping that mirrors real-world data (tables, lists, records).
- Simplifies passing multiple related values to functions.
- Forms the base for implementing higher data structures (stacks, queues, matrices).

Types of Collections Supported by C

The C language provides a limited but powerful set of built-in collection types. Unlike languages such as Python or Java, C does not have built-in dynamic lists — collections in C are constructed using the following core types:

Collection Type	Description	Example
Array	Fixed-size, ordered collection of elements of the *same* data type stored in contiguous memory.	`int a[5];`
Structure (struct)	Collection of elements of *different* data types grouped under one name.	`struct Student { int roll; float marks; };`
Union	Similar to structure, but all members share the same memory location.	`union Data { int i; float f; };`
String	A special case of a character array terminated by a null character.	`char name[20] = "Rahul";`
Pointer-based / Dynamic Arrays	Memory allocated at run time using `malloc()` / `calloc()` to build resizable collections.	`int *p = malloc(5*sizeof(int));`

Exam Tip: If asked "Which is the most fundamental/simplest collection in C?" — the answer is always the **Array**, since structures and unions are built using arrays and other primitive types as their foundation.

Advantages of Collections

- Efficient and organized storage of bulk data.
- Reduces code length through loop-based processing.
- Enables meaningful grouping of related real-world data.
- Improves readability and maintainability of programs.

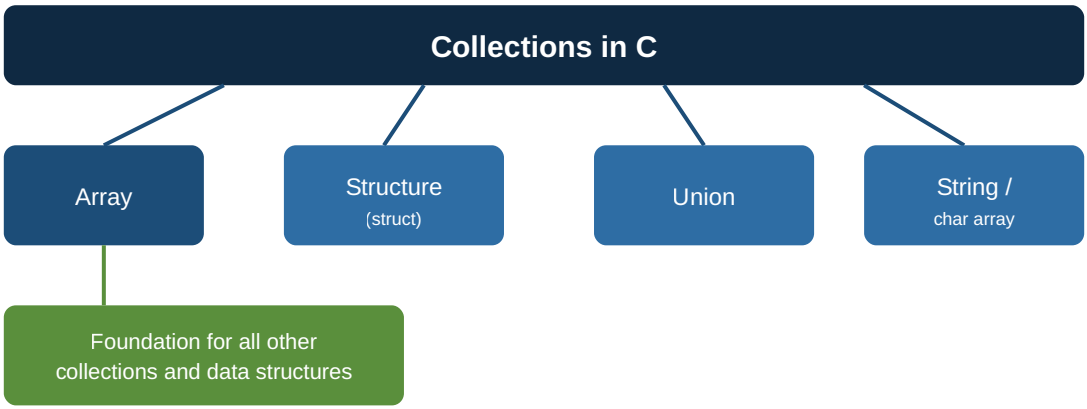


Fig 1.1 — Collection types supported by C and their relationship to arrays

Chapter Summary

- A collection groups related data under one name.
- C supports arrays, structures, unions, strings, and dynamic (pointer-based) collections.
- The array is the simplest and most fundamental collection type in C.

Chapter 2: Arrays and Their Features

2.1 Definition of Array

DEFINITION

An **array** is a collection of a fixed number of elements of the **same data type**, stored in **contiguous (continuous) memory locations**, where each element is **accessed using a common name combined with an index**.

Characteristics / Features of Arrays

Feature	Explanation
Homogeneous	All elements must be of the same data type.
Fixed size	Size is decided at declaration time (for static arrays) and cannot grow/shrink at run time.
Contiguous memory	Elements are stored in continuous, back-to-back memory locations.
Indexed access	Every element can be accessed directly using its index — no need to traverse from the start.
Single common name	All elements share one identifier; individual elements are distinguished only by index.
Random access	Any element can be accessed in constant time, O(1), using its index.

Advantages of Arrays

- Easy and fast access to elements using index (random access).
- Efficient for storing and processing large volumes of similar data.
- Works naturally with loops, reducing code repetition.
- Forms the base for implementing stacks, queues, matrices, and strings.

Limitations of Arrays

- Size is fixed at compile time; cannot be resized dynamically (in static arrays).
- Insertion and deletion of elements is costly (requires shifting elements).
- Memory may be wasted if the declared size exceeds the actual requirement.
- Must store elements of only one data type.

Applications of Arrays

- Storing marks, roll numbers, sales figures, sensor readings.
- Implementing matrices, strings, and lookup/hash tables.
- Base structure for stacks, queues, and heaps.
- Used in searching and sorting algorithms.

int marks[5] = {80, 90, 75, 60, 95};

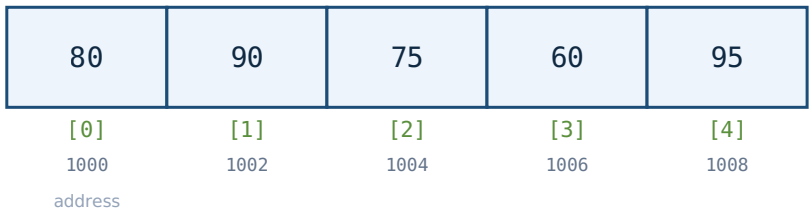


Fig 2.1 — Array structure in memory: contiguous cells, each 2 bytes apart (int assumed 2 bytes here for spacing clarity)

Exam Tip: "Contiguous memory allocation" and "homogeneous elements" are the two most commonly tested keywords when defining an array.

Chapter Summary

- Array = fixed-size, homogeneous, contiguous, indexed collection.
- Main advantage: random access in O(1) time. Main limitation: fixed size, costly insertion/deletion.

Chapter 3: Indexing Concepts

3.1 Concept of Indexing

DEFINITION

Indexing is the mechanism of assigning a unique positional number to each element of an array so that it can be individually accessed. In C, indexing always starts from **0** and goes up to **(size − 1)**.

Index Variable

An **index variable** is a variable (usually of integer type) used to hold the position number of the element being referred to at any given moment. It acts as a "cursor" that moves across the array, and is most commonly declared and controlled by a loop so that the entire array can be traversed systematically.

SYNTAX

```
arrayName[indexVariable]

Example:
int i;
for (i = 0; i < 5; i++)
    printf("%d ", marks[i]);   /* i is the index variable */
```

The index variable need not always be a plain variable — it can be any **expression** that evaluates to an integer, such as `marks[i+1]`, `marks[2*i]`, or `marks[i-1]`. The value of this expression at the time of access decides exactly which element is picked.

Index Type

RULE

The expression used inside `[ ]` must always evaluate to an **integral type** — `int`, `short`, `long`, `unsigned int`, or even `char` (since `char` is internally stored and treated as a small integer in C).

Index Expression	Valid?	Reason
`a[3]`	Yes	Integer constant
`a[i]`	Yes	`i` declared as `int`
`a['A']`	Yes	Character constant; internally treated as integer 65
`a[i+2]`	Yes	Expression evaluates to an integer
`a[2.5]`	No (compiler error/warning)	Floating-point value is not a valid index type
`a[-1]`	Syntactically allowed, logically invalid	Negative index falls outside the array's valid range

Because the index type must be integral, the compiler internally uses it directly in the address computation `Base + index × size of element` — a floating or fractional index would make this formula meaningless, which is why C disallows it.

Positional Value of a Member of Array Collection

The **positional value** (also called the index value) of an element denotes its exact order/location within the array, counted from the very first element. The first element has positional value 0, the second has positional value 1, and so on. This numbering scheme, where counting begins at 0 rather than 1, is called **zero-based indexing**, and it is the standard followed throughout the C language.

KEY RELATIONSHIP

Positional value of an element = number of elements that precede (come before) it in the array. This positional value is exactly what is used as the index, and it is also what gets multiplied by the element size while computing the element's memory address (see Chapter 4).

Element	Position (Human count)	Index (C notation)
1st element	1st	0
2nd element	2nd	1
3rd element	3rd	2
n-th element	n-th	n − 1

int a[6] = {12, 45, 7, 89, 34, 21};



Fig 3.1 — Indexed array representation with zero-based indexing

SOLVED EXAMPLE

Given `int a[6] = {12, 45, 7, 89, 34, 21};`, find the value at index 3 and the index of value 34.

```
Value at a[3] = 89
Index of value 34 → position 5th element → index 4
```

COMMON ERROR

Confusing "size" with "highest index". If an array has size `n`, the valid indices are `0` to `n-1`, NOT `1` to `n`.

Chapter Summary

- Indexing gives each element a unique position number, starting at 0.

- Index variable: holds current position; Index type: must be integral.
- Positional value of the n-th element = n − 1 (zero-based indexing).

Chapter 4: Dimensions and Size of Arrays

4.1 Concept of Dimension

DEFINITION

The **dimension** of an array is the number of indices (indices is just the plural of index) that must be supplied to reach a single element of that array. Each dimension corresponds to one "direction" along which the array's data is organized, and hence to one pair of square brackets in the element's notation.

A 1-D array lays its elements out along a single line, so one index is enough to point to any element. A 2-D array organizes elements into rows and columns (like a grid or table), so two indices — one for the row, one for the column — are needed. In general, an array with **N dimensions** needs exactly **N indices** to identify one element.

Dimension	Common Name	Indices Needed	Element Notation	Declaration Pattern
1	One-dimensional (1-D) array	1 (e.g. `i`)	`a[i]`	`type a[n];`
2	Two-dimensional (2-D) array / Matrix	2 (e.g. `i, j`)	`a[i][j]`	`type a[r][c];`
3	Three-dimensional (3-D) array	3 (e.g. `i, j, k`)	`a[i][j][k]`	`type a[p][q][r];`

Exam Tip: One-line definition to remember — **"Dimension = number of subscripts (index brackets) used to access an element of an array."**

4.2 Concept of Size

DEFINITION

The **size** of an array is the total number of elements it is declared to hold. Unlike dimension, which describes the array's *structure* (how many indices), size describes the array's *capacity* (how many values it can store).

SYNTAX

```
dataType arrayName[SIZE];

Example:
int week[7];      /* size = 7 elements */
float temp[10];   /* size = 10 elements */
```

RULE

The value written inside `[ ]` to fix the size must be a **positive integer constant expression**. It cannot be zero, negative, or a fractional value, and in standard C it is normally decided at compile time.

BEST PRACTICE

Use a named constant instead of a bare number for the size, e.g. `#define DAYS 7` followed by `int week[DAYS];`. If the size needs to change later, only one line needs to be edited.

4.3 How Dimension and Size Together Decide Memory

FORMULA

Total memory required (in bytes) = **Total number of elements × size of one element (in bytes)**

For a 1-D array, the total number of elements is simply its declared size. For a multi-dimensional array, the total number of elements is the **product of the sizes along each dimension** — for example, `int a[3][4];` has `3 × 4 = 12` elements in total. Whatever the dimension, this entire block of memory is reserved **contiguously, at compile time** — this is called **static memory allocation**.

Declaration	Dimension	Total Elements	Bytes/Element	Total Memory
`int a[10];`	1	10	4	40 bytes
`float f[7];`	1	7	4	28 bytes
`int m[3][4];`	2	3 × 4 = 12	4	48 bytes
`double d[10];`	1	10	8	80 bytes

SOLVED EXAMPLE

For `float temp[7];` with `sizeof(float) = 4` bytes:

```
Total memory = 7 × 4 = 28 bytes
```

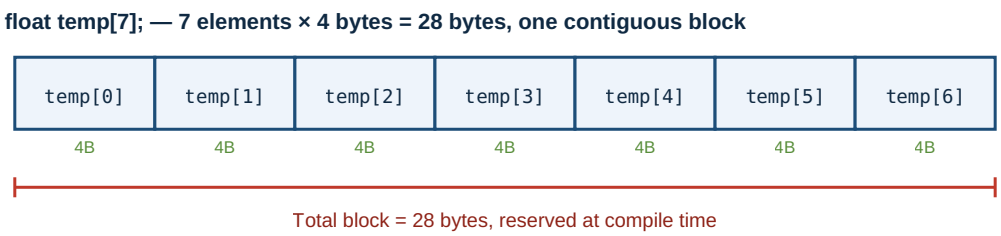


Fig 4.1 — Memory layout of `float temp[7];`

Dimension vs. Size — Quick Comparison

Aspect	Dimension	Size
Answers the question	"How many indices do I need?"	"How many elements fit inside?"
Written as	Number of bracket pairs — `[ ]`, `[ ] [ ]`, ...	The number(s) written inside the bracket(s)
Example	`a[i][j]` → dimension = 2	`int a[5];` → size = 5
Determines	The shape of the array (line, grid, cube, ...)	The total memory the array consumes

Chapter Summary

- Dimension = number of indices needed to access an element (1-D → one, 2-D → two, and so on).
- Size = total number of elements an array can hold; must be a positive integer constant fixed at declaration.
- Total memory (bytes) = total elements × size of one element; reserved as one contiguous block at compile time.
- For multi-dimensional arrays, total elements = product of the sizes along each dimension.

Chapter 5: Array Declaration and Syntax

5.1 C Syntax for Array Declaration

GENERAL SYNTAX

```
dataType arrayName[sizeOfArray];

Example:
int marks[5];       /* array of 5 integers   */
float price[10];    /* array of 10 floats    */
char name[20];      /* array of 20 characters */
```

Array Name and Its Type

- The **array name** acts as a symbolic constant that represents the **base address** (address of the first element) of the array.
- The **type of the array** is determined by the data type of the elements it stores (e.g. `int marks[5]` is of type "array of int").
- An array name cannot be re-assigned to point elsewhere (unlike a pointer variable) — it is a constant reference to its own memory block.

Naming Conventions

- Must follow standard C identifier rules (letters, digits, underscore; cannot start with a digit).
- Should be descriptive of the data being stored, e.g. `studentMarks`, `temperature`.
- Cannot be a C keyword (e.g. `int`, `float`).

Declaration Examples

EXAMPLES

```
int roll[50];        // array to hold 50 roll numbers
float salary[20];    // array to hold 20 salary values
char grade[100];     // array of 100 characters
double reading[8];   // array of 8 sensor readings
```

Syntax Summary Table

Component	Role	Example
`dataType`	Type of every element (all elements share this type)	`int`, `float`
`arrayName`	Symbolic name / base address of the array	`marks`
`[sizeOfArray]`	Total number of elements the array can hold	`[5]`
`;`	Statement terminator	—

COMMON ERROR

Writing the size using a variable in older (pre-C99) standard without support for VLAs, or forgetting that array size must be a positive integer constant expression in strict declarations.

Chapter Summary

- Syntax: `dataType arrayName[size];`
- Array name = base address; array type = type of its elements.

Chapter 6: Accessing Array Elements

6.1 Referring to Individual Elements

An individual element is referred to by writing the array name followed by the desired index in square brackets: `arrayName[index]`. This expression can be used anywhere a normal variable of that type can be used — in assignments, expressions, or function calls.

SYNTAX

```
arrayName[index] = value;      /* writing to an element   */
variable = arrayName[index];   /* reading from an element */
```

6.2 Entering Data Into an Array

DEFINITION

Entering data means storing values into an array's elements — typically by looping through the indices from `0` to `size−1` and reading one value into each element using `scanf`.

SYNTAX

```
for (i = 0; i < size; i++)
    scanf("%d", &arrayName[i]);   /* & is required — scanf needs the address */
```

EXAMPLE — ENTERING MARKS INTO AN ARRAY

```
#include <stdio.h>
int main() {
    int marks[5], i;
    printf("Enter 5 marks:\n");
    for (i = 0; i < 5; i++)
        scanf("%d", &marks[i]);     /* entering data */
    return 0;
}
```

OUTPUT

```
Enter 5 marks: 80 90 75 60 95
```

6.3 Reading Data From an Array

DEFINITION

Reading data means retrieving the value already stored at a given index — for display, computation, or passing elsewhere in the program — typically using `printf` inside a loop.

SYNTAX

```
for (i = 0; i < size; i++)
    printf("%d ", arrayName[i]);   /* no & needed — arr[i] is already a value */
```

EXAMPLE — READING AND DISPLAYING THE MARKS ARRAY

```
#include <stdio.h>
int main() {
    int marks[5] = {80, 90, 75, 60, 95}, i;
    printf("You entered:\n");
    for (i = 0; i < 5; i++)
        printf("%d ", marks[i]);     /* reading data */
    return 0;
}
```

OUTPUT

```
You entered: 80 90 75 60 95
```

INPUT OPERATION

Uses `scanf("%d", &arr[i]);` — the **address-of operator (&)** is required because `scanf` needs the memory address to store the value.

OUTPUT OPERATION

Uses `printf("%d", arr[i]);` — no `&` is needed because `arr[i]` already evaluates to the value itself, not its address.

Exam Tip: A very common exam trap is asking why `&` is used with `scanf` but not with `printf` for array elements — remember: `scanf` needs an address to write into; `printf` needs a value to read from.

Chapter Summary

- Elements accessed as `arrayName[index]`.
- Data entered using `scanf` with `&`; data read/displayed using `printf` without `&`.

Chapter 7: Array Initialization

7.1 Concept of Initialization

DEFINITION

Array initialization means assigning initial values to array elements at the time of declaration itself, using a comma-separated **list of initializers** enclosed in curly braces `{ }` .

SYNTAX

```
dataType arrayName[size] = {value0, value1, value2, ..., valueN-1};

Example:
int marks[5] = {80, 90, 75, 60, 95};
```

Partial Initialization

If fewer values are supplied than the declared size, the remaining elements are automatically initialized to **zero** (for numeric types).

EXAMPLE

```
int a[5] = {10, 20};   /* a = {10, 20, 0, 0, 0} */
```

Size Option in Initialization

If the size is **omitted**, the compiler automatically determines the array size from the number of initializers provided.

EXAMPLE

```
int a[] = {5, 10, 15, 20};  /* size automatically becomes 4 */
```

Initialization Style	Example	Resulting Array
Full initialization	`int a[3] = {1,2,3};`	{1, 2, 3}
Partial initialization	`int a[5] = {1,2};`	{1, 2, 0, 0, 0}
Size omitted	`int a[] = {1,2,3,4};`	{1,2,3,4}, size = 4
All-zero initialization	`int a[5] = {0};`	{0,0,0,0,0}

BEST PRACTICES

- Prefer `int a[] = {...};` when the value list itself defines size, to avoid mismatch errors.
- Always initialize arrays before use to avoid garbage (undefined) values.
- Use `{0}` to quickly initialize an entire numeric array to zero.

COMMON ERROR

Supplying more initializers than the declared size causes a compile-time error: `int a[3] = {1,2,3,4};` → "excess elements in array initializer".

Chapter Summary

- Initializer list assigns values at declaration time.
- Missing values default to 0; size can be omitted and inferred from the list.

Chapter 8: Bounds Checking

8.1 Meaning of Bounds Checking

DEFINITION

Bounds checking is the process of verifying that an array index lies within its valid range, i.e., between `0` and `size − 1`, before accessing that element.

IMPORTANT FACT

C does **NOT** perform automatic bounds checking at compile time or run time. Accessing an out-of-range index does not raise an error in C — it results in **undefined behaviour**, since the compiler simply computes an address and accesses whatever memory happens to be there.

Array Index Limits

For an array declared as `dataType arr[N];`, the only valid indices are `0, 1, 2, ..., N-1`. Using index `N` or higher, or a negative index, accesses memory **outside** the array's allocated block.

Out-of-Bound Access — Example

DEFINITION

Out-of-bound access means using an index that lies outside the array's valid range (i.e., less than `0` or greater than `N-1` for an array of size `N`) to read or write an element. Since C does not check this, the program ends up accessing memory that does not actually belong to the array.

EXAMPLE (UNDEFINED BEHAVIOUR)

```
int a[5] = {1,2,3,4,5};
printf("%d", a[10]);   /* undefined behaviour — may print garbage,
                          crash, or corrupt other memory  */
```

Common Errors Due to Missing Bounds Checking

Error	Cause	Example

Off-by-one error	Looping with `i <= N` instead of `i < N`	`int a[5];`
		`for (i = 0; i <= 5; i++)  /* wrong: runs i=0..5, but a[5] is invalid */`
		`    a[i] = 0;`

Buffer overflow	Writing beyond array limits, corrupting adjacent memory/ variables	`int a[5] = {0};` `a[7] = 100;  /* writes past the array, may overwrite another variable */`
Garbage output	Reading an uninitialized or out-of-range element	`int a[5];` `printf("%d", a[2]);  /* a[2] was never set — prints leftover/garbage value */`
Segmentation fault	Accessing memory far outside the process's allocated space	`int a[5];` `printf("%d", a[100000]);  /* index far outside → likely crashes the program */`

Debugging Tips

- Always use the condition `i < size` (not `<=`) in loops.
- Store the array size in a named constant and reuse it consistently.
- Use tools like compiler warnings, `-Wall`, or run-time sanitizers to catch overflow bugs.

Exam Tip: A guaranteed exam question: "Does C provide automatic array bounds checking?" — Answer: **No**. This is a key difference from languages like Java, which throw `ArrayIndexOutOfBoundsException`.

Chapter Summary

- Valid indices: 0 to size−1. C performs no automatic bounds checking.
- Out-of-bound access leads to undefined behaviour, not a compiler/runtime error.

Chapter 9: Two-Dimensional Arrays

9.1 Concept and Representation

DEFINITION

A **two-dimensional (2-D) array** is an array of arrays — a collection of elements organized in **rows and columns**, accessed using two indices: one for the row and one for the column. It is commonly used to represent **matrices**.

Declaration Syntax

SYNTAX

```
dataType arrayName[rowSize][columnSize];

Example:
int matrix[3][4];   /* 3 rows, 4 columns → 12 elements total */
```

Initialization Methods

DEFINITION

Initializing a 2-D array means assigning starting values to its elements at the time of declaration itself, using values enclosed in curly braces `{ }`. Because a 2-D array is really an "array of rows," its initializer list is usually written as a set of **nested braces** — one inner `{ }` group per row.

Method 1 — Row-wise (nested-brace) Initialization

Each row is written as its own `{ }` group, and the groups are separated by commas. This is the clearest and most recommended style, since it visually mirrors the rows and columns of the matrix.

EXAMPLE

```
int matrix[2][3] = {
    {1, 2, 3},     /* row 0 */
    {4, 5, 6}      /* row 1 */
};
```

Method 2 — Flat (linear) Initialization

The inner braces can be dropped altogether, and all values listed in one flat sequence. The compiler fills the array in **row-major order** — it completely fills row 0 first, then moves on to row 1, and so on.

EXAMPLE

```
int matrix[2][3] = {1, 2, 3, 4, 5, 6};
/* Same result as Method 1 — row 0 = {1,2,3}, row 1 = {4,5,6} */
```

Method 3 — Omitting the Row Size

If the array is initialized at declaration, the **row size can be left blank** and the compiler will count the number of `{ }` groups to determine it automatically. The **column size, however, can never be omitted** — the compiler needs it to know how many elements make up one row, so it can correctly compute the offset for every row.

EXAMPLE

```
int matrix[][3] = {{1, 2, 3}, {4, 5, 6}};
/* Row size automatically becomes 2 (two {} groups); column size stays 3 */
```

Method 4 — Partial Initialization

If a row's braces contain fewer values than the declared column size, the remaining elements of **that row** are automatically set to `0` — exactly as with 1-D arrays.

EXAMPLE

```
int matrix[2][3] = {
    {1, 2},     /* becomes {1, 2, 0} */
    {4}         /* becomes {4, 0, 0} */
};
```

Method 5 — All-zero Initialization

A single `{0}` can be used to initialize every element of the array to zero, regardless of its dimensions.

EXAMPLE

```
int matrix[2][3] = {0};   /* all 6 elements become 0 */
```

Method	Example	Result
Row-wise (nested braces)	`{{1,2,3},{4,5,6}}`	Row 0 = {1,2,3}, Row 1 = {4,5,6}
Flat (linear) list	`{1,2,3,4,5,6}`	Filled row-major — same result as above
Row size omitted	`int m[][3]={{1,2,3},{4,5,6}};`	Row count inferred as 2 from the number of groups
Partial initialization	`{{1,2},{4}}` for `m[2][3]`	{1,2,0}, {4,0,0}
All-zero shortcut	`{0}`	Every element set to 0

COMMON ERROR

Trying to omit the **column** size, e.g. `int matrix[2][] = {{1,2,3},{4,5,6}};`, causes a compile-time error. Only the **row** size may be left blank — the column size is always mandatory.

Accessing Elements

DEFINITION

Accessing an element of a 2-D array means referring to it using two **zero-based** indices — a row index and a column index. For `int matrix[rowSize][columnSize];`, valid row indices run from `0` to `rowSize−1`, and valid column indices run from `0` to `columnSize−1`. Using either index outside its valid range is out-of-bound access (Chapter 8) — C does not check this automatically for 2-D arrays either.

SYNTAX

```
arrayName[rowIndex][columnIndex]

Example:
matrix[1][2]   /* element in 2nd row, 3rd column */
```

Input and Output Operations

DEFINITION

Input and output on a 2-D array means entering values into, and displaying values from, every element of the matrix. Since two indices are involved, this always requires **two nested loops** — the outer loop steps through the rows, and the inner loop steps through the columns of the current row — using `scanf` with `&m[i][j]` to enter data, and `printf` with `m[i][j]` to display it.

EXAMPLE — READING AND PRINTING A 2×3 MATRIX

```
#include <stdio.h>
int main() {
    int m[2][3], i, j;
    printf("Enter 6 elements:\n");
    for (i = 0; i < 2; i++)
        for (j = 0; j < 3; j++)
            scanf("%d", &m[i][j]);

    printf("Matrix:\n");
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 3; j++)
            printf("%d\t", m[i][j]);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
Enter 6 elements: 1 2 3 4 5 6 Matrix: 1 2 3 4 5 6
```

Matrix Representation

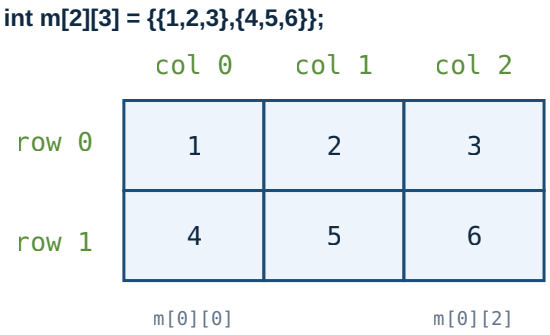


Fig 9.1 — Logical (matrix) view of a 2-D array

Memory Organization of 2-D Arrays

Physical memory is always **linear (one-dimensional)**. A 2-D array is actually stored as a single contiguous block, arranged **row by row** — this scheme is called **row-major order** (the default in C).

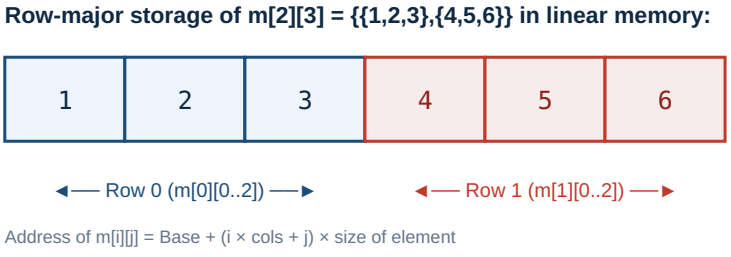


Fig 9.2 — Row-major memory layout: the entire 2-D array is one continuous block

SOLVED EXAMPLE — ADDRESS CALCULATION

For `int m[3][4]`, base address 2000, `sizeof(int)=4`, find address of `m[2][1]`.

```
Address = Base + (row × totalCols + col) × elementSize
        = 2000 + (2 × 4 + 1) × 4
        = 2000 + 9 × 4
        = 2000 + 36 = 2036
```

Exam Tip: Row-major order is C's default; column-major order is used by Fortran. This comparison is a frequent one-mark question.

Chapter Summary

- 2-D array: `dataType arr[rows][cols];`, accessed as `arr[i][j]`.
- Physically stored linearly in row-major order in C.
- Address formula: Base + (i × cols + j) × element size.

Chapter 10: Dimension and Nesting

10.1 Concept of Dimensions Revisited

As established earlier, the **dimension** of an array is the number of indices needed to locate an element. A 1-D array is a single list; a 2-D array is a "list of lists" (rows, each of which is itself a 1-D array); a 3-D array is a "list of lists of lists," and so on.

10.2 Nested Loops and Arrays

Because a 2-D array is conceptually an array of arrays, it is naturally processed using **nested loops** — an outer loop for the row index and an inner loop for the column index, mirroring the two dimensions exactly.

SYNTAX — NESTED LOOP TRAVERSAL

```
for (i = 0; i < rows; i++) {        /* outer loop → dimension 1 (row) */
    for (j = 0; j < cols; j++) {    /* inner loop → dimension 2 (col) */
        printf("%d ", arr[i][j]);
    }
}
```

Similarities Between Dimension and Nesting

Aspect	Array Dimension	Loop Nesting
Structure	Each extra dimension adds one more index, e.g. `[i][j][k]`	Each extra dimension needs one more nested loop level
Order	Outer-most dimension = row (slowest-changing)	Outer-most loop corresponds to the outer-most (row) dimension
Depth relationship	An N-D array has N indices	Traversing an N-D array fully needs N nested loops
Conceptual view	2-D array = array of 1-D arrays	Nested loop = loop containing another loop, mirroring "array of arrays"
Access pattern	Innermost index changes fastest (row-major)	Innermost loop variable changes fastest

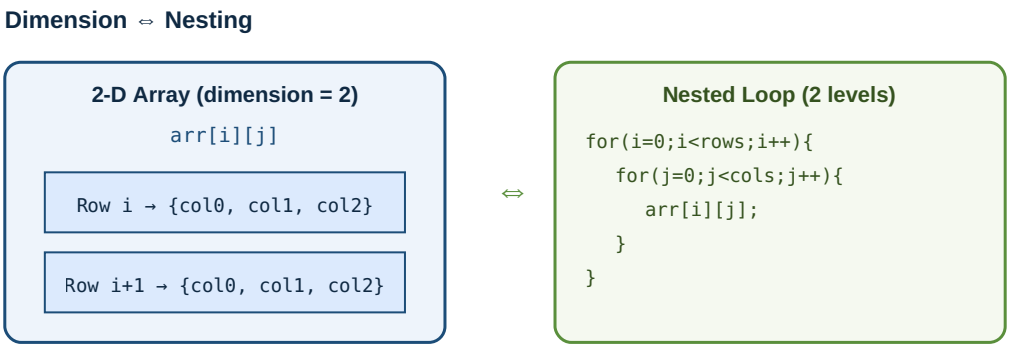


Fig 10.1 — Each dimension of an array corresponds to one level of loop nesting

EXAMPLE — 3-D ARRAY NEEDS 3 NESTED LOOPS

```
int cube[2][3][4];
for (i = 0; i < 2; i++)
    for (j = 0; j < 3; j++)
        for (k = 0; k < 4; k++)
            scanf("%d", &cube[i][j][k]);   /* 3 dimensions → 3 nested loops */
```

Exam Tip: Rule of thumb for exams: **"Number of dimensions = Number of nested loops = Number of index brackets."**

Chapter Summary

- An N-dimensional array requires N indices and is naturally traversed using N nested loops.
- Dimension and nesting mirror each other structurally: each extra dimension = one more loop level.

Chapter 11: Programming Examples

11.1 Traversing an Array

```c
#include <stdio.h>
int main() {
    int a[5] = {10, 20, 30, 40, 50}, i;
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

OUTPUT

```
10 20 30 40 50
```

11.2 Searching an Element (Linear Search)

```c
#include <stdio.h>
int main() {
    int a[5] = {10, 20, 30, 40, 50}, key = 30, i, found = 0;
    for (i = 0; i < 5; i++) {
        if (a[i] == key) { found = 1; break; }
    }
    if (found)
        printf("Element found at index %d", i);
    else
        printf("Element not found");
    return 0;
}
```

OUTPUT

```
Element found at index 2
```

11.3 Finding Maximum and Minimum

```c
#include <stdio.h>
int main() {
    int a[6] = {12, 45, 7, 89, 34, 21}, i, max = a[0], min = a[0];
    for (i = 1; i < 6; i++) {
        if (a[i] > max) max = a[i];
        if (a[i] < min) min = a[i];
    }
    printf("Max = %d, Min = %d", max, min);
    return 0;
}
```

OUTPUT

```
Max = 89, Min = 7
```

11.4 Sum and Average

```c
#include <stdio.h>
int main() {
    int a[5] = {80, 90, 75, 60, 95}, i, sum = 0;
    float avg;
    for (i = 0; i < 5; i++)
        sum += a[i];
    avg = sum / 5.0;
    printf("Sum = %d, Average = %.2f", sum, avg);
    return 0;
}
```

OUTPUT

```
Sum = 400, Average = 80.00
```

11.5 Matrix Addition

```c
#include <stdio.h>
int main() {
    int a[2][2] = {{1,2},{3,4}};
    int b[2][2] = {{5,6},{7,8}};
    int c[2][2], i, j;
    for (i = 0; i < 2; i++)
        for (j = 0; j < 2; j++)
            c[i][j] = a[i][j] + b[i][j];

    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++)
            printf("%d ", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
6 8 10 12
```

11.6 Matrix Multiplication

```c
#include <stdio.h>
int main() {
    int a[2][2] = {{1,2},{3,4}};
    int b[2][2] = {{5,6},{7,8}};
    int c[2][2], i, j, k;
    for (i = 0; i < 2; i++)
        for (j = 0; j < 2; j++) {
            c[i][j] = 0;
            for (k = 0; k < 2; k++)
                c[i][j] += a[i][k] * b[k][j];
        }

    for (i = 0; i < 2; i++) {
        for (j = 0; j < 2; j++)
            printf("%d ", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
19 22 43 50
```

Exam Tip: Matrix multiplication requires **three** nested loops (i, j, k) — since it involves two dimensions of the result plus one dimension for the summation index.

Chapter 12: Quick Revision Section

12.1 Important Definitions (One-Line)

Term	One-line Definition
Collection	A construct grouping multiple related data items under one name.
Array	Fixed-size, homogeneous, contiguous, indexed collection of elements.
Index	Positional number identifying an element's location in an array.
Index variable	Integer-type variable holding the current index during traversal.
Dimension	Number of indices needed to identify an array element.
Initializer list	Comma-separated values in `{ }` used to set initial array values.
Bounds checking	Verifying an index lies within 0 to size−1 (NOT automatic in C).
Row-major order	Storage scheme where 2-D array elements are stored row by row.

12.2 Key Concepts at a Glance

- Arrays are the most basic collection type in C; structures/unions build on top of primitive types.
- Indexing is zero-based: valid range is `0` to `size − 1`.
- Memory required = number of elements × size of each element.
- Array name = base address; cannot be reassigned.
- C performs **no automatic bounds checking** — out-of-range access is undefined behaviour.
- 2-D arrays are stored in row-major order; address = base + (i×cols + j) × element size.
- Number of array dimensions = number of nested loops needed to fully traverse it.

12.3 Syntax Summary

Operation	Syntax
1-D Declaration	`dataType arr[size];`
1-D Initialization	`dataType arr[size] = {v0, v1, ...};`
1-D Access	`arr[i]`
2-D Declaration	`dataType arr[rows][cols];`
2-D Initialization	`dataType arr[r][c] = {{...},{...}};`
2-D Access	`arr[i][j]`
Input	`scanf("%d", &arr[i]);`
Output	`printf("%d", arr[i]);`

12.4 Frequently Asked Exam Questions

Q1. Why is an array called a homogeneous collection?
Because all its elements must be of the same data type.

Q2. Does C support dynamic array resizing?
Not directly for static arrays; resizing requires dynamic memory functions like `malloc` / `realloc`.

Q3. What happens on out-of-bound array access in C?
It causes undefined behaviour since C does not perform automatic bounds checking.

Q4. What is the default storage order of 2-D arrays in C?
Row-major order — all elements of a row are stored contiguously before moving to the next row.

Q5. Why does the number of dimensions equal the number of nested loops?
Because each dimension needs its own index variable, and each index variable is naturally driven by its own loop level during traversal.

Q6. Can the row size be omitted in a 2-D array initialization, but not the column size?
Yes — the compiler needs the column count to compute the correct offset for each row, but can infer the row count from the number of initializer groups.

12.5 Common Mistakes in Array Programming

Mistake	Correct Practice
Using `i <= size` in loops	Use `i < size` since valid indices are 0 to size−1
Forgetting `&` in `scanf`	Always use `&arr[i]` with `scanf`
Assuming C checks array bounds	Manually validate indices before access
Mismatched initializer count and declared size	Match counts, or omit size and let the compiler infer it
Confusing dimension count with element count	Dimension = index count; size = total elements
Omitting column size in 2-D array declaration	Column size is mandatory; only row size can be omitted

One-Page Revision Notes

- **Collection** → groups related data (array, struct, union, string).
- **Array** → fixed size, same type, contiguous memory, index-based access.
- **Indexing** → zero-based; index variable is integral; index range 0 to n−1.
- **Size** → total bytes = elements × size of one element; address = base + i×size.
- **Declaration** → `type name[size];` ; array name = base address (constant).
- **Access** → `scanf("%d",&a[i])` to input, `printf("%d",a[i])` to output.
- **Initialization** → `{v0,v1,...}` ; missing values become 0; size can be omitted.
- **Bounds checking** → NOT automatic in C; programmer's responsibility.
- **2-D arrays** → `type name[r][c]` ; row-major storage; address = base+(i×c+j)×size.
- **Dimension** ⇔ **Nesting** → N dimensions require N indices and N nested loops.